

REM Assembly Language Programming Manual

REM project

Revision 1.0 — 2026-09-22

Document status and scope

This is the offline programming reference for the completed REM Linux guest and its **REM** 32-bit CPU. It assumes the native editor, shell, assembler, linker, binutils, GCC, Make, headers, libraries, and persistent filesystem described by the REM Programmer's Guide are available in the guest.

The repository also contains a legacy **MyEmulator** 8-bit-data/16-bit-address firmware target. Its instruction set is intentionally different and is documented separately in `docs/instruction-set.md`, `docs/architecture.md`, and `docs/assembler.md`. Do not combine examples from those documents with the REM examples in this manual.

The architecture contract in this manual is the programming interface. Release evidence, implementation discrepancies, and acceptance tests are maintained separately in `REM_DOCUMENTATION_VERIFICATION.md`; they are not prerequisites for following the user workflows in this manual.

1. Programmer-visible architecture

REM is a 32-bit computer with a little-endian, byte-addressed processor and fixed-width instructions. General integer values, virtual addresses, physical addresses, and instruction words are 32 bits. Arithmetic wraps modulo ($2^{\{32\}}$) unless an instruction raises an exception.

Instructions are fixed-width 32-bit words, stored little-endian, and must be 4-byte aligned. Normal sequential execution increments PC by four.

1.1 General registers

There are sixteen encoded general-purpose registers. Register-field values 16–31 are reserved and are rejected by the decoder.

Register	Alias	Meaning	ABI treatment
r0	zero	Reads as zero; writes are discarded	Constant
r1–r4	—	General values and first four arguments	Caller-saved
r5–r12	—	General values	Callee-saved
r13	sp	Banked stack pointer	Callee-managed
r14	lr	Link/return register	Preserved when a function calls
r15	—	General temporary; GCC uses it as a frame-base register in relevant builds	Caller-saved

PC is a separate architectural register. SR is a separate status/control register. r13 selects USP in user mode and SSP in supervisor mode.

1.2 Status register and privilege

SR bit assignments are:

Bits	Name	Meaning
0	CF	Carry for addition; unsigned borrow occurred for subtraction
1	OF	Signed two's-complement overflow
2–4	IPL	Interrupt priority level, 0–7
5	S	Supervisor mode when set
6–31	reserved	Read as zero; writes have no effect

There are two execution modes. User mode cannot directly access supervisor state. Supervisor code can select the user and supervisor stack pointers

through system-register operations. An interrupt is accepted only when its level is greater than IPL; IPL=7 masks ordinary IRQs.

The current Linux port relies on the system registers SR, USP, SSP, VBR, PTBR, MMCR, TIME_LO, TIME_HI, TIMECMP_LO, TIMECMP_HI, TP, and DFSP. A 32-bit transfer never claims to read or write the 64-bit timer as one operation.

1.3 Exceptions and interrupts

The current vector assignments are:

Vector	Cause
0	Illegal instruction
1	Privilege violation
2	Instruction alignment
3	Data alignment
4–5	Instruction page-not-present/protection
6–9	Load/store page-not-present/protection
10	Divide by zero
11	Arithmetic overflow
12	System call
13	Breakpoint
14	NMI
15	Double fault
16–22	IRQ1–IRQ7

The CPU builds an exception frame on the supervisor stack. Linux expands this into its software `pt_regs` frame before calling C code. General registers are not automatically preserved by the hardware; handlers must save registers they modify.

IRQ entry saves the next instruction address and complete pre-interrupt status. RFE/the Linux return path restores the saved state. Nested interrupts are possible only for higher-priority levels.

1.4 Memory and alignment

The virtual and physical address spaces are 32-bit. Loads and stores use a base register plus a signed 13-bit displacement. LB/LBU access one byte, LH/LHU two bytes, and LW four bytes. SB, SH, and SW store one, two, and four bytes respectively. Natural alignment is required for multi-byte accesses; misalignment raises the data-alignment exception.

The current development machine loads `-kernel` bytes at physical address zero. The Linux user linker script places user images near 0x02000000 and the kernel

port maintains the required mappings. These are distinct from the legacy 16-bit firmware map.

2. Instruction encoding

Every instruction occupies one 32-bit little-endian word. Bit 31 is the most significant bit of the decoded word.

```
R:      op[31:26] rd[25:21] ra[20:16] rb[15:11] fn[10:6] reserved[5:0]
I:      op[31:26] rd[25:21] ra[20:16] sub[15:12] imm[11:0]
MEM:    op[31:26] rt[25:21] base[20:16] size[15:13] disp[12:0]
B:      op[31:26] cond[25:23] ra[22:18] rb[17:13] disp[12:0]
J:      op[31:26] disp[25:0]
IND:    op[31:26] link[25] ra[24:20] reserved[19:0]
U:      op[31:26] rd[25:21] imm[20:1] reserved[0]
SYS:    op[31:26] sysop[25:22] reg[21:17] sysreg[16:11] reserved[10:0]
SYSCALL: op[31:26] immediate[25:0]
TLB:    op[31:26] page[25] ra[24:20] reserved[19:0]
CAS:    op[31:26] rd[25:21] rs[20:16] ra[15:11] reserved[10:0]
```

Primary opcode allocation:

Opcode	Family
0x00	R-format ALU and comparisons
0x01	Immediate ALU and shifts
0x02	Loads
0x03	Stores
0x04	Conditional branches
0x05	J
0x06	JAL
0x07	JR/JALR
0x08	LUI
0x09	System-register operations
0x0A	SYSCALL
0x0B	TLB flush
0x0C	Compare-and-swap
0x0D–0x3F	Reserved

The machine-readable source is `docs/myemulator2-encoding.json`. Reserved fields must remain zero. A malformed or reserved encoding raises an illegal instruction exception.

3. Instruction reference

The following list is the complete instruction set represented by the current REM encoding manifest.

3.1 Integer and logical operations

For every R-format instruction, `ra`, `rb`, and the old `SR.CF` are read before `rd` is written. Consequently, `add r1, r1, r2` is valid and uses the old value of `r1` for both the left operand and the destination. `r0` always reads as zero and discards writes. Arithmetic results are reduced modulo 2^{32} .

`CF` and `OF` are replaced by `add`, `adc`, `sub`, `sbc`, `addi`, and `subi`. For addition, `CF` is the carry out of bit 31; for subtraction, `CF=1` means an unsigned borrow occurred. `adc` computes `ra + rb + old_CF`; `sbc` computes `ra - rb - old_CF`. `OF` is signed two's-complement overflow for the complete operation. These instructions do not preserve the incoming flags. Multiply, divide, remainder, logic, shifts, rotates, and comparisons leave `CF` and `OF` unchanged. There are no implicit `N/Z` flags.

Instruction	Exact result	Operand restrictions and exceptions
<code>add rd, ra, rb</code>	<code>rd = ra + rb</code>	None; updates <code>CF</code> , <code>OF</code>
<code>adc rd, ra, rb</code>	<code>rd = ra + rb + old_CF</code>	None; updates <code>CF</code> , <code>OF</code>
<code>sub rd, ra, rb</code>	<code>rd = ra - rb</code>	None; updates <code>CF</code> , <code>OF</code>
<code>sbc rd, ra, rb</code>	<code>rd = ra - rb - old_CF</code>	None; updates <code>CF</code> , <code>OF</code>
<code>mul rd, ra, rb</code>	Low 32 bits of signed <code>ra * rb</code>	<code>CF</code> , <code>OF</code> unchanged
<code>mulh rd, ra, rb</code>	High 32 signed bits of <code>ra * rb</code>	<code>CF</code> , <code>OF</code> unchanged
<code>mulhu rd, ra, rb</code>	High 32 unsigned bits of <code>ra * rb</code>	<code>CF</code> , <code>OF</code> unchanged
<code>div rd, ra, rb</code>	Signed quotient, truncated toward zero	<code>rb=0</code> raises vector 10; <code>0x80000000 / 0xffffffff</code> raises vector 11
<code>divu rd, ra, rb</code>	Unsigned quotient	<code>rb=0</code> raises vector 10
<code>rem rd, ra, rb</code>	<code>ra - div(ra, rb) * rb</code>	Same signed exceptions as <code>div</code>
<code>remu rd, ra, rb</code>	Unsigned remainder	<code>rb=0</code> raises vector 10
<code>and rd, ra, rb</code>	<code>rd = ra & rb</code>	None
<code>or rd, ra, rb</code>	<code>rd = ra rb</code>	None
<code>xor rd, ra, rb</code>	<code>rd = ra ^ rb</code>	None

Instruction	Exact result	Operand restrictions and exceptions
<code>not rd, ra, r0</code>	<code>rd = ~ra</code>	<code>rb</code> must be <code>r0</code> ; otherwise illegal instruction
<code>sll rd, ra, rb</code>	<code>rd = ra << (rb & 31)</code>	Logical left shift
<code>srl rd, ra, rb</code>	<code>rd = ra >> (rb & 31)</code>	Logical right shift
<code>sra rd, ra, rb</code>	Arithmetic <code>rd = signed(ra) >> (rb & 31)</code>	Sign-extends shifted-in bits
<code>rol rd, ra, rb</code>	Rotate <code>ra</code> left by <code>rb & 31</code>	Count zero returns <code>ra</code>
<code>ror rd, ra, rb</code>	Rotate <code>ra</code> right by <code>rb & 31</code>	Count zero returns <code>ra</code>
<code>seq rd, ra, rb</code>	<code>rd = (ra == rb) ? 1 : 0</code>	No flags
<code>sne rd, ra, rb</code>	<code>rd = (ra != rb) ? 1 : 0</code>	No flags
<code>slt rd, ra, rb</code>	<code>rd = signed(ra) < signed(rb) ? 1 : 0</code>	No flags
<code>sge rd, ra, rb</code>	<code>rd = signed(ra) >= signed(rb) ? 1 : 0</code>	No flags
<code>sltu rd, ra, rb</code>	<code>rd = ra < rb ? 1 : 0</code>	Unsigned comparison
<code>sgeu rd, ra, rb</code>	<code>rd = ra >= rb ? 1 : 0</code>	Unsigned comparison

Individual R-format entries

All entries below use the R-format encoding `[31:26]=0`, `[25:21]=rd`, `[20:16]=ra`, `[15:11]=rb`, `[10:6]=function`, `[5:0]=0`. The function values are `add=0`, `adc=1`, `sub=2`, `sbc=3`, `mul=4`, `mulh=5`, `mulhu=6`, `div=7`, `divu=8`, `rem=9`, `remu=10`, `and=11`, `or=12`, `xor=13`, `not=14`, `sll=15`, `srl=16`, `sra=17`, `rol=18`, `ror=19`, `seq=20`, `sne=21`, `slt=22`, `sge=23`, `sltu=24`, and `sgeu=25`. Every operand is a register; an invalid function or nonzero reserved bits traps as illegal instruction.

- **`add rd, ra, rb`** — unsigned modulo- 2^{32} addition. Sets `CF` to carry out and `OF` to signed overflow. Example: `add r1, r1, r2`.
- **`adc rd, ra, rb`** — adds `ra`, `rb`, and the old `CF`; replaces both arithmetic flags with the complete result. Example: `adc r2, r2, r4`.
- **`sub rd, ra, rb`** — computes `ra-rb`; sets `CF` when unsigned borrow occurs and sets signed `OF`. Example: `sub r1, r1, r2`.
- **`sbc rd, ra, rb`** — computes `ra-rb-old_CF`; the old flag is an incoming borrow, and the new `CF` reports borrow from the complete subtraction. Example: `sbc r2, r2, r4`.

- **mul rd, ra, rb** — low 32 bits of signed multiplication; flags unchanged.
- **mulh rd, ra, rb** — high 32 bits of signed multiplication; flags unchanged.
- **mulhu rd, ra, rb** — high 32 bits of unsigned multiplication; flags unchanged.
- **div rd, ra, rb** — signed quotient truncated toward zero. Zero divisor raises vector 10; signed minimum divided by -1 raises vector 11.
- **divu rd, ra, rb** — unsigned quotient; zero divisor raises vector 10.
- **rem rd, ra, rb** — signed remainder using the corresponding quotient rule and the same two exceptions as **div**.
- **remu rd, ra, rb** — unsigned remainder; zero divisor raises vector 10.
- **and rd, ra, rb, or rd, ra, rb, and xor rd, ra, rb** — bitwise operations; flags unchanged.
- **not rd, ra, r0** — bitwise complement. The third operand must be **r0**; any other register encoding is illegal.
- **sll rd, ra, rb, srl rd, ra, rb, and sra rd, ra, rb** — shift by **rb** & 31, logically left, logically right, or arithmetically right respectively; flags unchanged.
- **rol rd, ra, rb and ror rd, ra, rb** — rotate by **rb** & 31; a zero count returns **ra**; flags unchanged.
- **seq rd, ra, rb and sne rd, ra, rb** — write 1 for equal/not-equal and 0 otherwise; flags unchanged.
- **slt rd, ra, rb and sge rd, ra, rb** — signed less-than and greater-than-or-equal; write 1 or 0 and leave flags unchanged.
- **sltu rd, ra, rb and sgeu rd, ra, rb** — unsigned equivalents; write 1 or 0 and leave flags unchanged.

Register overlap is valid for every R-format entry unless an entry states a restriction. Operands are read before **rd** is written.

Immediate forms are **addi**, **subi**, **andi**, **ori**, **xori**, **slli**, **srli**, and **srai**. **addi**/**subi** sign-extend the 12-bit immediate; **andi**/**ori**/**xori** zero-extend it. Immediate shifts use **imm[4:0]**; the upper immediate bits must be zero or the instruction is illegal. The following is a complete carry-chain example:

```
# Add a 64-bit value in r2:r1 to r4:r3, low words first.
add    r1, r1, r3          # sets CF from the low-word addition
adc    r2, r2, r4          # consumes that CF
```

The I-format encoding is **[31:26]=1**, **[25:21]=rd**, **[20:16]=ra**, **[15:12]=subop**, **[11:0]=imm**. The subops are **addi=0**, **subi=1**, **andi=2**, **ori=3**, **xori=4**, **slli=5**, **srli=6**, and **srai=7**.

- **addi rd, ra, imm** and **subi rd, ra, imm** use a signed 12-bit immediate in the range -2048 through 2047 and update CF/OF.
- **andi rd, ra, imm**, **ori rd, ra, imm**, and **xori rd, ra, imm** use a zero-extended 12-bit mask and leave flags unchanged.
- **slli rd, ra, count**, **srli rd, ra, count**, and **srai rd, ra, count**

use a zero-extended count from 0 through 31; bits 11:5 must be zero. They leave flags unchanged.

Every immediate form permits `rd == ra`; `r0` remains a constant zero.

3.2 Loads and stores

```
lb  r1, 0(r5)      # sign-extended byte
lbu r1, 1(r5)      # zero-extended byte
lh  r2, 2(r5)      # sign-extended halfword
lhu r2, 4(r5)      # zero-extended halfword
lw  r3, 8(r5)      # 32-bit word
sb  r3, 0(r5)
sh  r3, 2(r5)
sw  r3, 4(r5)
```

The effective address is $(\text{base} + \text{sign_extend}(\text{disp13})) \bmod 2^{32}$. There is no implicit base-register update. A load writes `rt`; a store reads `rt`. The memory encoding is $[31:26]=2$ for loads or 3 for stores, $[25:21]=\text{rt}$, $[20:16]=\text{base}$, $[15:13]=\text{size}$, and a signed displacement in $[12:0]$. Size codes are byte 0, halfword 1, and word 2; load subcodes select signed (`lb`, `lh`) or zero-extended (`lbu`, `lhu`) values. Word loads and stores require addresses divisible by four, and halfword operations require addresses divisible by two. A fault occurs before a store changes memory.

3.3 Control transfer

`J` and `JAL` encode a signed 26-bit displacement in instruction words. Conditional branches encode a signed 13-bit displacement and compare two registers:

```
beq r1, r2, equal
bne r1, r2, different
blt r1, r2, less
bge r1, r2, not_less
bltu r1, r2, unsigned_less
bgeu r1, r2, unsigned_not_less
j    loop
jal  function
jr   r14
jalr r5          # link in lr; target address in r5
```

For a control-transfer instruction at `P`, the target is:

$$\text{target} = (P + 4) + \text{sign_extend}(\text{encoded_displacement}) * 4$$

Targets must be 4-byte aligned. `JAL` writes the following instruction address to `r14` before transferring. `JR` transfers without linking; `JALR` links and transfers through a register. The branch encoding is $[31:26]=4$, condition in $[25:23]$,

registers in [22:18] and [17:13], and signed displacement in [12:0]. Conditions are `beq=0`, `bne=1`, `blt=2`, `bge=3`, `bltu=4`, and `bgeu=5`. `J` uses opcode 5 and `JAL` opcode 6 with a signed 26-bit displacement. `JR/JALR` use opcode 7 and the target register in [24:20]; `JALR` sets the link bit [25]. `jalr` therefore has one GAS operand: `jalr r5`.

3.4 System, TLB, and atomic operations

`mfsr rd, sysreg` reads a system register and `mtsr sysreg, rs` writes one. `rfe` returns from an exception frame. `halt` stops the virtual CPU and `break` enters the breakpoint exception path. `syscall immediate` enters the Linux syscall exception; the immediate is currently reserved for kernel diagnostics and does not replace the syscall number in `r1`.

`tlbflush all` invalidates the complete translation cache. The page form invalidates the translation associated with the supplied page/register according to the QEMU target implementation.

`cas rd, rs, 0(ra)`

CAS uses the CAS encoding (`rd`, `rs`, `ra`, and zero reserved bits) and performs one naturally aligned 32-bit atomic operation:

```
old = MEM32[ra]
if old == rd:
    MEM32[ra] = rs
rd = old
```

The comparison operand is the original value of `rd`; the returned value is also written to `rd`. Register overlap is therefore well-defined because all register operands are read before the destination write. The `0(ra)` address form is mandatory; nonzero displacements are rejected. `ra` must be word-aligned and the address must be readable and writable. An alignment or MMU fault is raised before the memory update. CAS does not modify `CF` or `OF` and is strongly ordered with surrounding memory operations; there are no weaker acquire/release modes or fence instructions. For example, this loop increments a shared word only when no other agent changed it:

```
retry:
    lw      r1, 0(r5)           # expected value
    add     r3, r1, r0          # CAS rd starts with the expected value
    addi    r2, r1, 1           # desired value
    cas     r3, r2, 0(r5)       # r3 becomes the observed old value
    bne     r3, r1, retry       # retry unless observed == expected
```

The Linux kernel and libc may provide higher-level atomic interfaces, but they must not be assumed to add memory-ordering guarantees beyond this architectural contract.

4. GNU assembler and object files

The canonical GNU target is `myemulator2-elf`. Typical commands are:

```
myemulator2-elf-as -o hello.o hello.S
myemulator2-elf-ld -T toolchain/userspace/myemulator2-user.ld \
    -o hello hello.o
myemulator2-elf-objdump -dr hello
myemulator2-elf-readelf -h -l -S -r hello
```

Source uses GNU syntax. Registers are `r0`–`r15`, with `zero`, `sp`, and `lr` aliases. `#` begins a comment. Memory operands are `disp(base)`. Standard sections and symbol directives are used:

```
.section .text
.global _start
.type _start,@function
.align 2
```

The project ABI supports `.text`, `.rodata`, `.data`, `.bss`, `.section`, `.global/.globl`, `.local`, `.type`, `.size`, `.align`, `.balign`, `.p2align`, `.byte`, `.short`, `.word`, `.long`, `.ascii`, `.asciz`, `.space`, `.zero`, `.fill`, `.equ`, `.set`, `.include`, and ordinary GAS conditional and macro facilities. The target assembler accepts these directives directly; unknown directives and malformed expressions are reported as assembly errors.

`.word` is a 32-bit little-endian value for REM. Use `.short` for 16-bit data and `.byte` for individual bytes. Alignment directives align the location counter; they do not change the CPU instruction width.

Expressions, labels, macros, and conditional assembly

GAS evaluates constants and relocatable expressions at assembly or link time. The target-specific address pair is:

```
lui    r5, buffer
ori    r5, r5, buffer
```

Local numeric labels are valid (`1f` means the next `1:`, and `1b` the previous one). Named symbols can be made visible with `.global/.globl`, kept local with `.local`, and annotated with `.type` and `.size`. External references are ordinary undefined symbols resolved by the linker:

```
.extern puts
.global main
main:
    addi    sp, sp, -16
    sw      lr, 12(sp)
    lui     r1, message
    ori     r1, r1, message
```

```

jal    puts
li     r1, 0
lw     lr, 12(sp)
addi   sp, sp, 16
ret

```

The installed GAS supports its normal `.equ`, `.set`, `.include`, `.macro`, `.endm`, `.if`, `.ifdef`, `.ifndef`, `.else`, and `.endif` facilities. The target port does not add a separate macro language. Keep reusable definitions in an include file and pass its directory with `-I`:

```
myemulator2-elf-as -I include -alh=program.lst -o program.o program.S
```

Use `.struct/.struct`-style symbolic offsets or `.equ` constants for structures; GAS does not create a runtime object layout. Arrays are normally defined with `.rept`, `.fill`, or `.space`. Numeric labels are preferable inside macros because named labels must remain unique after expansion.

Generate a source listing with `-alh/--listing`, inspect relocations before linking with `objdump -dr`, and inspect the final ELF with:

```

myemulator2-elf-readelf -h -l -S -s -r program.elf
myemulator2-elf-objdump -drwC program.elf

```

The listing and inspection commands are verified binutils workflow; the target-specific support boundary is recorded in `docs/REM_DOCUMENTATION_VERIFICATION.md`.

The project assembler recognizes `ret`, `call`, and `li` as the documented pseudo-operations. A `li` that cannot fit in one immediate is expanded into a `lui/ori` sequence. `call symbol` is a link-and-transfer operation. No legacy MyEmulator `push/pop` spelling should be assumed for this target unless it is present in the current GAS build.

5. Relocations and linking

The initial ABI uses ELF32 little-endian RELA relocations:

Number	Name	Use
0	R_MYEMU_NONE	No operation
1	R_MYEMU_32	32-bit data address/value
2	R_MYEMU_BRANCH13	Conditional branch displacement
3	R_MYEMU_JUMP26	J/JAL displacement
4	R_MYEMU_HI20	LUI high part
5	R_MYEMU_LO12	ALU-immediate low part

For a symbol `S`, addend `A`, and relocation place `P`, branch and jump relocations compute $D = S + A - (P + 4)$, require 4-byte divisibility, and encode

D/4 in the signed field. The linker rejects overflow and malformed instruction templates.

To construct an arbitrary address:

```
lui  r5, message
ori  r5, r5, message
```

The user linker script publishes the ELF program headers inside a loadable segment because musl startup uses `AT_PHDR`:

```
text PT_LOAD FILEHDR PHDRS FLAGS(7)
entry: _start
user image base: 0x02000000 + SIZEOF_HEADERS
```

Dynamic linking, GOT/PLT conventions, shared libraries, and dynamic TLS are not part of the stable current ABI.

6. ABI and calling convention

The stack grows downward and is 16-byte aligned at call boundaries. There is no red zone. The first four arguments are in `r1–r4`; later arguments are on the stack. `r1` returns a 32-bit result. A 64-bit result uses `r1:r2`, low word first.

`r1–r4` and `r15` are caller-saved. `r5–r12` are callee-saved. `sp` is callee-managed. A function that executes a nested call must preserve its incoming `lr`.

```
.text
.global add_two
.type add_two,@function
add_two:
    add r1, r1, r2      # result in r1
    ret                # jr r14
.size add_two, .-add_two
```

The ABI does not yet freeze aggregate argument/return rules, variadic register save areas, unwind metadata, stack canaries, floating-point register passing, or dynamic TLS. C programs should use the compiler-generated convention until those areas are formally released.

7. Linux executable startup

The canonical static executable is ELF32 little-endian with project-local machine value `0xF2E2`. `_start` is the ELF entry point. A normal C program then enters the musl startup sequence, which prepares TLS and calls `main`.

Freestanding programs can provide `_start` directly:

```

.section .text
.global _start
.type _start,@function
_start:
    # program body
    li r1, 94          # exit_group
    li r2, 0
    syscall 0

```

Standalone `_start` programs receive no documented `argc/argv` register interface. Use a hosted `main(int argc, char **argv, char **envp)` when a program needs command-line arguments or environment variables; the musl startup code constructs those arguments before calling `main`. A freestanding `_start` program may use fixed arguments or a separately documented kernel interface, but must not assume an ARM, RISC-V, or x86 initial-stack layout.

The tested crt0 pattern is:

```

.global _start
_start:
    call main
    add r2, r1, r0
    li r1, 94
    syscall 0
1:    j 1b

```

8. Linux syscall ABI

The Linux entry instruction is `syscall`. `r1` holds the syscall number and receives the return value. Arguments occupy `r2` through `r7`, allowing six arguments. Negative returns are Linux `-errno` values.

The following syscalls are part of the REM Linux programming interface. For every call, place the number in `r1`, arguments in `r2` through `r7`, execute `syscall 0`, and read the signed result from `r1`. A negative result is `-errno`; successful byte-count and file-descriptor results are nonnegative.

Number	Name	Current implementation
17	<code>getcwd</code>	Yes
23	<code>dup</code>	Yes
24	<code>dup3</code>	Yes
25	<code>fcntl</code>	Yes
29	<code>ioctl</code>	Yes
34	<code>mkdirat</code>	Yes
35	<code>unlinkat</code>	Yes
38	<code>renameat</code>	Yes

Number	Name	Current implementation
49	chdir	Yes
56	openat	Yes
57	close	Yes
59	pipe2	Yes
61	getdents64	Yes
62	lseek	Yes
63	read	Yes
64	write	Yes
66	writew	Yes
74	fsync	Yes
79	newfstatat	Yes
80	fstat	Yes
93	exit	Yes
94	exit_group	Yes
96	set_tid_address	Yes
99	set_robust_list	Yes
101	nanosleep	Yes
124	sched_yield	Yes
134	rt_sigaction	Yes
135	rt_sigprocmask	Yes
160	uname	Yes
172	getpid	Yes
173	getppid	Yes
175	geteuid	Yes
178	gettid	Yes
214	brk	Yes
215	munmap	Yes
220	clone	Yes
221	execve	Yes
222	mmap	Yes
260	wait4	Yes
403	clock_gettime (32-bit time structure)	Yes
422	futex	Yes

The following argument contracts use the Linux 32-bit target layouts:

Call	Arguments after the number	Result
read 63	fd, buf, count	bytes read; 0 is EOF
write 64	fd, buf, count	bytes written
writew 66	fd, iov, iovcnt	total bytes
openat 56	dirfd, path, flags, mode	file descriptor

Call	Arguments after the number	Result
close 57	fd	0
lseek 62	fd, offset, whence	new offset
fstat 80	fd, statbuf	0
newfstatat 79	dirfd, path, statbuf, flags	0
getdents64 61	fd, buffer, buffer_size	bytes of records
fsync 74	fd	0
mkdirat 34	dirfd, path, mode	0
unlinkat 35	dirfd, path, flags	0
renameat 38	old_dirfd, old_path, new_dirfd, new_path	0
chdir 49	path	0
getcwd 17	buffer, size	buffer address
mmap 222	addr, length, prot, flags, fd, page_offset	mapped address
munmap 215	addr, length	0
brk 214	address	new/current break
pipe2 59	fd_pair, flags	0
dup 23	oldfd	new descriptor
dup3 24	oldfd, newfd, flags	new descriptor
fcntl 25	fd, command, argument	command result
ioctl 29	fd, request, argument	request result
exit 93 / exit_group 94	status	does not return
nanosleep 101	request, remaining	0
clock_gettime 403	clock_id, timespec32	0
sched_yield 124	none	0
getpid 172, getppid 173, geteuid 175, gettid 178	none	identifier
uname 160	struct new_utsname *	0
execve 221	path, argv, envp	only on error
clone 220	flags, child_stack, parent_tid, child_tid, tls	child/parent PID
wait4 260	pid, status, options, rusage	child PID

Call	Arguments after the number	Result
futex 422	uaddr, operation, value, timeout, uaddr2, value3	operation result
set_tid_address 96	tidptr	thread ID
set_robust_list 99	head, length	0
rt_sigaction 134	signum, act, oldact, sigset_size	0
rt_sigprocmask 135	how, set, oldset, sigset_size	0

Useful constants are AT_FDCWD=-100, O_RDONLY=0, O_WRONLY=1, O_RDWR=2, O_CREAT=64, O_EXCL=128, O_TRUNC=512, O_APPEND=1024, SEEK_SET=0, SEEK_CUR=1, SEEK_END=2, PROT_READ=1, PROT_WRITE=2, PROT_EXEC=4, MAP_SHARED=1, MAP_PRIVATE=2, MAP_ANONYMOUS=32, STDIN_FILENO=0, STDOUT_FILENO=1, and STDERR_FILENO=2.

The `struct old_timespec32` used by `nanosleep` and `clock_gettime` contains two signed 32-bit fields: `tv_sec` at offset 0 and `tv_nsec` at offset 4. The `struct iovec` used by `writew` contains a 32-bit pointer at offset 0 and a 32-bit byte count at offset 4. The `struct stat` layout is target-header dependent and should be accessed through a C helper until its complete userspace layout is frozen. `struct linux_dirent64` is defined in the directory-listing exercise and may be parsed without headers.

Example direct write:

```
.section .text
.global _start
_start:
    li r1, 64
    li r2, 1
    lui r3, message
    ori r3, r3, message
    li r4, message_end-message
    syscall 0
    li r1, 94
    li r2, 0
    syscall 0
.section .rodata
message: .ascii "Hello from REM\n"
message_end:
```

Reads and writes may be partial and may return negative errors. Robust code checks `r1`, retries only when the operation's documented error is interruptible,

and never treats a negative result as a byte count.

9. Calling C and libc

The active hosted C library work targets musl. The project also has a small minilibc used by bootstrap tests. Static linking is the supported model. Dynamic linking and a stable shared-library ABI are not complete.

The verified C libc regression exercises:

```
puts("libc-start");
malloc(32); realloc(...); calloc(...);
memset; strlen; strcmp; snprintf; printf;
uname; open; fstat; read; write; close;
mkdir; stat; nanosleep;
```

The source is `toolchain/examples/linux-libc-regression.c`. Do not infer that every POSIX or ISO C function is available from that test. Link with the project compiler driver and linker script supplied by the toolchain build, rather than invoking a host linker.

An assembly implementation of a non-variadic `main` follows the normal C ABI:

```
.text
.global main
.type main,@function
main:
    # r1 is the return value
    li r1, 0
    ret
.size main, .-main
```

The following is the complete assembly-to-libc boundary currently supported as documentation: assembly supplies `main`, the project `crt0` performs musl initialization, and the compiler driver links the static libc:

```
.section .rodata
message:
    .asciz "hello from assembly via libc"

.text
.global main
.type main,@function
.extern puts
main:
    addi    sp, sp, -16
    sw      lr, 12(sp)
    lui     r1, message
```

```

ori    r1, r1, message
jal    puts                # puts(const char *)
li     r1, 0                # return 0 from main
lw     lr, 12(sp)
addi   sp, sp, 16
ret
.size main, .-main

```

Build this only with the hosted REM Linux compiler driver and its musl sysroot; myemulator2-elf-ld alone does not provide crt0 or libc:

```
myemulator2-linux-musl-gcc -static hello.S -o hello
```

This example establishes the integer argument register convention and the non-variadic call path. A direct assembly call to `printf` is **not currently verified**: the variadic register-save-area and aggregate ABI are explicitly unfrozen. The C regression's `printf`, `snprintf`, `malloc`, `realloc`, `strlen`, `strcmp`, and file operations prove only the compiler-generated C path. Until a dedicated variadic ABI test passes, assembly must use a small C wrapper for formatted output rather than guessing register placement.

10. Memory management and data structures

Use `lui/ori` to form a full pointer, then use a base register and signed displacement for members:

```

lui    r5, object
ori    r5, r5, object
lw     r1, 0(r5)           # first 32-bit field
lh     r2, 4(r5)           # signed 16-bit field
sb     r3, 6(r5)           # byte field

```

Keep stack frames 16-byte aligned. A four-byte local can be allocated with `addi sp, sp, -16`, stored at `0(sp)`, and released with `addi sp, sp, 16`. The compiler backend emits explicit stack updates and ordinary `lw/sw`; there is no architectural post-increment addressing mode.

`mmap`, `munmap`, and `brk` exist in the current Linux dispatcher, but their complete userspace memory-management contract is still tied to the evolving Linux port. Prefer libc allocation functions in hosted programs.

11. Strings, character data, and I/O

String literals use `.ascii` plus an explicit terminator or `.asciz`:

```

.section .rodata
prompt: .asciz "value: "

```

The CPU has no string instructions. Implement loops with `lb/lbu`, compare bytes in registers, and advance a pointer with `addi`. For terminal I/O, Linux `read` and `write` use descriptors 0, 1, and 2. Early PID 1 console startup has a kernel fallback for `stdin/stdout/stderr` because descriptors may not yet be inherited.

12. Debugging and disassembly

Useful tools:

```
myemulator2-elf-objdump -drwC program
myemulator2-elf-readelf -h -l -S -s -r program
myemulator2-elf-nm program
```

QEMU can run the `myemulator32` machine with a kernel and serial console:

```
qemu-system-myemulator32 -M myemulator32 -m 16M \
    -kernel .linux-build/build/vmlinux \
    -nographic -serial stdio
```

The tracked tests are the executable behavioral specification: `tests/run-myemulator32-cpu-tests.py`, `tests/run-myemulator32-integer-tests.py`, `tests/run-myemulator32-system-tests.py`, `toolchain/tests/test-isa.py`, and the Linux tests under `toolchain/scripts/test-linux-*.py`.

When investigating a fault, record the fault vector, PC, SR, SP, LR, and info address. A user page fault is not evidence that an instruction encoding is wrong; it may be a Linux MMU, ELF, or userspace-layout failure.

13. YOUR FIRST COMPLETE REM ASSEMBLY PROGRAM

This walkthrough uses only tools installed in the REM guest. Start in an empty persistent directory:

```
mkdir -p "$HOME/src/hello"
cd "$HOME/src/hello"
vi hello.S
```

Enter this complete program:

```
.section .text
.global _start
.type _start,@function
_start:
    li    r1, 64
    li    r2, 1
    lui   r3, message
    ori   r3, r3, message
```

```

        li      r4, message_end-message
        syscall 0
        li      r1, 94
        li      r2, 0
        syscall 0
1:      j        1b

.section .rodata
message:
        .ascii  "Hello from REM assembly\n"
message_end:

```

Assemble and inspect the relocatable object:

```

myemulator2-elf-as -o hello.o hello.S
myemulator2-elf-readelf -h -S -s hello.o
myemulator2-elf-objdump -dr hello.o

```

Link, inspect, and run it:

```

ldscript="$(myemulator2-linux-musl-gcc -print-file-name=myemulator2-user.ld)"
myemulator2-elf-ld -T "$ldscript" -o hello hello.o
myemulator2-elf-readelf -h -l -s hello
myemulator2-elf-objdump -d hello
./hello
printf 'exit status: %s\n' "$?"

```

The output is `Hello from REM assembly` followed by a newline and the exit status is 0. To make a change, edit `hello.S`, rerun the two build commands, and execute the new image. For a deliberate diagnostic, change `syscall 0` to `syscall bad`; GAS reports the invalid expression and does not create a valid object. Keep the source and executable under the persistent directory; they remain available after a clean REM restart.

13.1 Calling a non-variadic libc function

The hosted entry point `main` receives the normal C arguments and returns its status in `r1`. This example calls `puts`, which takes one pointer argument:

```

.section .rodata
message:
        .asciz  "Hello from REM libc"

.section .text
.global main
.type main,@function
.extern puts
main:
        addi    sp, sp, -16

```

```

        sw        lr, 12(sp)
        lui       r1, message
        ori       r1, r1, message
        jal       puts
        li        r1, 0
        lw        lr, 12(sp)
        addi      sp, sp, 16
        ret
.size main, .-main

```

Build it with the native C driver so that the musl startup files and libc are selected:

```

myemulator2-elf-as -o puts.o puts.S
myemulator2-linux-musl-gcc -static -o puts puts.o
./puts

```

The program prints Hello from REM libc. The first argument is in `r1`; `jal` writes the return address to `lr`, and `ret` returns from `main`. Variadic functions such as `printf` are not covered by this example.

14. Practical complete examples

14.1 Freestanding exit

```

.section .text
.global _start
_start:
        li r1, 94
        li r2, 0
        syscall 0
1:      j 1b

```

14.2 Function call and callee-saved register

```

.text
.global function
function:
        addi sp, sp, -16
        sw   r5, 0(sp)
        add  r5, r1, r0
        add  r1, r5, r2
        lw   r5, 0(sp)
        addi sp, sp, 16
        ret

```

14.3 Linux echo loop

This complete program copies standard input to standard output until end of file. It handles short writes and exits with status 1 if a system call fails.

```
.section .text
.global _start
.type _start,@function
_start:
read_again:
    li    r1, 63           # read
    li    r2, 0            # stdin
    lui   r3, buffer
    ori   r3, r3, buffer
    li    r4, 4096
    syscall 0
    blt   r1, r0, fail
    beq   r1, r0, success
    add   r4, r1, r0        # bytes still to write
    lui   r3, buffer
    ori   r3, r3, buffer
write_again:
    li    r1, 64           # write
    li    r2, 1            # stdout
    syscall 0
    blt   r1, r0, fail
    beq   r1, r0, fail      # zero progress would loop forever
    sub   r4, r4, r1
    add   r3, r3, r1
    bne   r4, r0, write_again
    j     read_again

success:
    li    r1, 94           # exit_group
    li    r2, 0
    syscall 0

fail:
    li    r1, 94
    li    r2, 1
    syscall 0
1:      j     1b

.section .bss
.align 2
buffer:
```

```
.space 4096
```

14.4 C calling assembly

C:

```
extern int add_two(int, int);
int main(void) { return add_two(20, 22) != 42; }
```

Assembly:

```
.text
.global add_two
.type add_two,@function
add_two:
    add r1, r1, r2
    ret
.size add_two, .-add_two
```

Build using the project GCC driver and the target sysroot; do not use the host compiler to assemble this source.

15. Programming exercises

These exercises are deliberately independent. Pick one, write the program, test it under REM Linux, and move on when it is useful to do so. They assume the instruction reference, the syscall ABI in section 8, the calling convention in section 6, and the GAS workflow in section 4. They do not require floating point, dynamic linking, undocumented syscalls, or direct assembly calls to variadic libc functions.

The exercises are grouped by the kind of work involved, not as a course. Each exercise states the observable contract before giving hints. Do not open the separate solutions directory until you have a program you are prepared to compare.

Group A — Register and memory programming

15.1 Reverse a string in place — moderate

Objective: Reverse a null-terminated ASCII string without allocating a second string. The terminator must remain at the same address.

Input/output: A pointer in `r1`; return the same pointer in `r1`. For "REM" the bytes become "MER".

Use: `lbu`, `sb`, `addi`, `beq`, `bne`, and a two-pointer loop.

Constraints: Work for an empty string, preserve `r5–r12`, and do not call libc. Test "", "A", "AB", "racecar", and a string containing spaces.

Edge cases: Empty input, odd/even lengths, and bytes with the high bit set.

Extension: Add a length-limited variant that never reads beyond a supplied maximum.

15.2 Signed integer to decimal ASCII — moderate

Objective: Convert a signed 32-bit integer to a caller-provided buffer, including a minus sign and terminating zero.

Input/output: `r1=value`, `r2=buffer`, `r3=capacity`; return length in `r1`, or -1 if the buffer is too small. -2147483648 must be supported.

Use: Division/remainder, signed comparisons, stores, and reverse-order digit generation.

Constraints: No libc and no 64-bit arithmetic. Document whether the returned length excludes the terminator.

Tests: 0 -> "0", 7 -> "7", -17 -> "-17", 2147483647, and -2147483648; test capacities 1, 2, 4, and exact-fit sizes.

Edge cases: Minimum signed value, zero capacity, and failed conversion.

Extension: Add an arbitrary radix from 2 through 16.

15.3 Hexadecimal parser with overflow detection — moderate

Objective: Parse an optional 0x-prefixed unsigned hexadecimal string.

Input/output: `r1=pointer`, `r2=end pointer`; return value in `r1` and a status in `r2` (0 success, 1 invalid character, 2 overflow).

Use: `1bu`, comparisons, shifts, `sltu`, and the pre-multiply overflow test value `> (UINT32_MAX-digit)/16`.

Tests: "", "0", "0xdeadBEEF", "ffffffff", "100000000", "12g4", and a string with a trailing newline.

Constraints: Do not silently accept signs or whitespace unless you document and test that extension.

Extension: Accept separators and return the address of the first error.

15.4 Population count — easy

Objective: Return the number of one bits in a 32-bit value.

Input/output: `r1=value`, return `r1` in the range 0–32.

Use: A shift-and-test loop or `x & (x-1)`; no table or libc.

Tests: 0 -> 0, 1 -> 1, 0xffffffff -> 32, 0x80000000 -> 1, 0xaaaaaaaa -> 16.

Edge cases: Ensure the loop terminates for zero and does not use an unimplemented instruction.

Extension: Count leading or trailing zeroes and define the zero result.

15.5 Signed integer sort — moderate

Objective: Sort an in-memory array of signed 32-bit integers ascending.

Input/output: `r1=base`, `r2=count`; sort in place and return the base.

Use: `lw`, `sw`, signed comparison, indexed-address construction, and callee-saved registers.

Tests: empty, one element, already sorted, reverse sorted, duplicates, `INT_MIN`, and `INT_MAX`.

Constraints: Use a simple quadratic algorithm first; do not hide the comparison in a `libc` call.

Extension: Add a descending flag and count comparisons.

15.6 Circular byte buffer — moderate

Objective: Implement `put`, `get`, `full`, and `empty` operations for a fixed capacity byte ring.

Input/output: Define a structure containing storage pointer, capacity, head, tail, and count. Return a distinct full/empty status without corrupting data.

Use: loads/stores, remainder or explicit wraparound, and `CAS` only if you choose to make a single-producer/single-consumer experiment.

Tests: fill to capacity, remove all, wrap twice, attempt `put` when full, and `get` when empty.

Constraints: Begin single-threaded; document that no lock-free guarantee is provided by the basic version.

Extension: Add line buffering or a blocking syscall wrapper.

Group B — Linux and terminal programming

15.7 Hexadecimal file dump — moderate

Objective: Read a file in blocks and print offsets, hexadecimal bytes, and printable ASCII.

Input/output: Filename is `argv[1]`; each row contains a fixed offset, up to 16 bytes, and an ASCII column replacing non-printable bytes with `..`

Use: `openat`, `read`, `write`, `close`, integer formatting, and the documented negative-error convention.

Tests: empty file, 1 byte, 16 bytes, 17 bytes, and binary data including zero and 0xff.

Constraints: Handle short reads and partial writes; do not assume the file fits in memory.

Extension: Add -C canonical format and a maximum byte count.

15.8 First differing byte — moderate

Objective: Compare two files and report the first differing offset.

Output: Print `equal` and exit 0 for identical files; otherwise print the offset and both byte values, then exit nonzero.

Use: `openat`, `read`, `close`, buffering, and unsigned byte comparison.

Tests: identical empty files, one file a prefix, difference at offset 0, difference across a buffer boundary, and different lengths.

Edge cases: Short reads, EOF on one descriptor, and read errors.

Extension: Report all differing ranges rather than only the first.

15.9 Line-oriented text search — involved

Objective: Search a file for a literal byte string and print matching line numbers and lines.

Input: `filename` and search string from `argv`; no regular expressions.

Use: Buffered `read`, line state, substring matching, and output formatting. Use only syscalls whose availability is listed in section 8.

Tests: match at line start, middle, end, overlapping matches, no match, empty search string, and a final unterminated line.

Constraints: Define maximum line length or implement spill storage.

Extension: Add case-insensitive ASCII matching and a count-only option.

15.10 Text statistics — involved

Objective: Count bytes, lines, words, and the frequency of every ASCII character in one pass over a file.

Output: A stable report with 64-bit-safe counters represented as high/low words if necessary.

Use: byte classification, a 256-entry table, buffered file I/O, and the integer conversion routine from section 15.2.

Tests: empty input, whitespace-only input, mixed line endings, UTF-8 bytes treated as bytes, and a final line without newline.

Constraints: State your word definition and apply it consistently.

Extension: Add the most frequent byte and a histogram mode.

15.11 Interactive hexadecimal/decimal/binary calculator — involved

Objective: Read commands from the terminal and evaluate addition, subtraction, AND, OR, and XOR.

Input/output: Accept expressions such as `0x10 + 7`, print a hexadecimal and decimal result, and report syntax/overflow errors without exiting.

Use: line input, the hexadecimal parser from section 15.3, operator dispatch, and output formatting. Do not call `printf` directly from assembly.

Tests: each operator, negative results, zero, malformed numbers, missing operands, and overflow.

Extension: Add shifts, parentheses, and a `quit` command.

15.12 Terminal memory inspector — involved

Objective: Display a user-owned buffer in hex and ASCII and allow a command to modify one byte.

Input/output: Commands select an offset and byte value; reject offsets outside the buffer and display the changed row.

Use: `read`, `write`, `lbu`, `sb`, address arithmetic, and command parsing.

Constraints: Inspect only a buffer allocated by the program. Do not attempt arbitrary process-memory access.

Tests: display empty/partial rows, modify first/last byte, invalid offset, invalid hex, and repeated edits.

Extension: Add word views and a save-to-file command.

15.13 Line editor — advanced

Objective: Implement insertion, deletion, left/right movement, redraw, and saving text to a file.

Input/output: Maintain an editable line and cursor; `Ctrl-D` deletes, arrow keys move, Enter saves or accepts according to your documented UI.

Use: terminal `read/write`, byte buffers, cursor arithmetic, and `openat/write` for saving.

Constraints: Define a maximum line length and handle partial terminal input. Do not assume `termios` support unless verified on the target.

Tests: insert at beginning/middle/end, delete at boundaries, move beyond boundaries, empty save, and full-buffer rejection.

Extension: Multiple lines and a command mode.

15.14 Directory listing — advanced

Objective: List a directory using the supported directory syscalls, displaying names and record types.

Use: `openat`, `getdents64`, `close`, and the following Linux `linux_dirent64` record layout:

offset	size	field
0	8	<code>d_ino</code> (unsigned 64-bit inode number)
8	8	<code>d_off</code> (signed 64-bit next-record offset)
16	2	<code>d_reclen</code> (unsigned 16-bit record length)
18	1	<code>d_type</code> (record type)
19	...	<code>d_name</code> (NUL-terminated name; record padding follows)

`d_reclen` includes the header, name, terminator, and padding. Advance by `d_reclen`, reject records shorter than 19 bytes or extending beyond the returned byte count, and stop at the first NUL in `d_name`. `DT_REG` is 8, `DT_DIR` is 4, `DT_LNK` is 10, and `DT_UNKNOWN` is 0.

Tests: empty directory, regular files, nested directories, long names, and repeated reads with partial records.

Constraints: Never assume one syscall returns a complete directory. Reject malformed record lengths to avoid an infinite loop.

Extension: Add sorting, `-a`, and stat information.

Group C — Larger programming challenges

15.15 Stack-based virtual machine — advanced

Objective: Execute a bytecode stream with push, arithmetic, comparisons, branches, and print-result instructions.

Input/output: Define an instruction encoding and report invalid opcodes, stack underflow, overflow, and branch-out-of-range errors.

Use: dispatch tables or comparisons, a data stack, bounds checks, and the non-variadic output path.

Tests: arithmetic program, comparison branch, nested operations, empty program, invalid opcode, and stack overflow.

Extension: Add calls, locals, and a bytecode assembler.

15.16 REM instruction disassembler — advanced

Objective: Decode 32-bit instruction words from a file or buffer into mnemonics, operands, and immediate fields.

Use: shifts, masks, signed extension, tables, file I/O, and the complete instruction encodings in section 2. The compact table in section 3 gives the field positions and restrictions needed to decode each family; the JSON manifest is not required.

Tests: one instruction from every family, reserved opcode, reserved fields, negative branch displacement, and truncated input.

Constraints: Match the manual's P+4 branch base and print unknown words as data rather than inventing mnemonics.

Extension: Add symbol names and a round-trip parser for the printed form.

15.17 Restricted two-pass assembler — advanced

Objective: Assemble a deliberately small language with labels, `li`, `add`, `lw`, `sw`, branches, `.word`, and `.ascii` into a flat image.

Input/output: Read a source file, report filename/line errors, resolve forward labels, and write bytes. ELF output is not required.

Use: file buffers, tokenization, symbol tables, two passes, and the instruction encodings.

Tests: forward/backward labels, duplicate/undefined labels, immediate overflow, malformed operands, data alignment, and comments.

Constraints: Document the subset and reject everything else.

Extension: Include files, numeric local labels, and a listing file.

15.18 Expression evaluator — advanced

Objective: Evaluate signed integer expressions with parentheses and operator precedence.

Input/output: Read one expression per line and print value or a precise syntax/error message.

Use: recursive descent or shunting-yard parsing, signed overflow checks, and the line I/O routines.

Tests: precedence, nested parentheses, unary minus, division by zero, missing parenthesis, adjacent operators, and minimum/maximum values.

Extension: Add symbols and hexadecimal/binary literals.

15.19 Tiny BASIC interpreter — very advanced

Objective: Support numbered lines, variables, arithmetic expressions, PRINT, INPUT, IF, GOTO, and RUN.

Input/output: Maintain a sorted program store and execute it only after RUN; report syntax and runtime errors without corrupting the program.

Use: line editor, tokenization, expression parser, sorted insertion, and controlled branches.

Tests: loops, conditional jumps, variable input, replacement/deletion of line numbers, empty program, undefined line, and division by zero.

Constraints: Define a finite line/program limit and report exhaustion.

Extension: FOR/NEXT, strings, and LIST.

15.20 Interactive REM machine-code monitor — very advanced

Objective: Build a monitor that displays a user-owned buffer, modifies bytes, disassembles instruction words, and executes selected predefined test routines.

Input/output: Commands such as `dump`, `edit`, `dis`, `run`, and `quit` must have documented syntax and errors.

Use: terminal I/O, the disassembler from 15.16, safe command parsing, and function calls through a fixed table of trusted routines.

Constraints: Never jump to arbitrary user input; execute only routines compiled into the program and validate all buffer ranges.

Tests: every command, invalid command, invalid range, reserved instruction, and a routine that returns a known value.

Extension: Breakpoints or single-step support using the documented debug interfaces, only after verifying them on the target.

Appendix C. Solutions and hints

The three hints for each exercise are intentionally here, rather than below the challenge, so the problem remains usable without immediately revealing an implementation.

Exercise	Hint 1 — general approach	Hint 2 — data struc- ture/algorithm	Hint 3 — REM-specific guidance
1	Use two pointers	Find the terminator, then swap inward	<code>1bu/sb</code> ; preserve callee-saved registers

Exercise	Hint 1 — general approach	Hint 2 — data struc- ture/algorithm	Hint 3 — REM-specific guidance
2	Generate digits backwards	Divide by ten into a temporary region	Handle <code>0x80000000</code> before negation
3	Validate then accumulate	Check <code>value > (max-digit)/16</code>	Use unsigned comparisons and <code>lbu</code>
4	Remove one set bit per loop	<code>x &= x-1</code>	<code>sub</code> and <code>and</code> avoid a 32-iteration minimum
5	Start with insertion sort	Shift larger elements right	<code>slt</code> is signed; scale indices by four
6	Track count explicitly	Head/tail/count distinguishes full/empty	Wrap with compare/subtract if division is unavailable
7	Read blocks and format rows	16-byte row plus output buffer	Loop on partial <code>read/write</code> ; errors are negative
8	Compare synchronized blocks	Keep offsets and remaining counts	Treat EOF as a third result, not a byte
9	Stream lines	Keep a rolling match index	Avoid direct <code>printf</code> ; write formatted buffers
10	Classify each byte once	256 counters plus <code>in_word</code>	Use <code>.space</code> and word-aligned counters
11	Separate lexer/parser/evaluator	Parse number, operator, number	Syscall input is byte-oriented in the bootstrap path
12	Keep address and length separate	Commands operate on offsets	Never expose arbitrary addresses to <code>lw/sw</code>

Exercise	Hint 1 — general approach	Hint 2 — data struc- ture/algorithm	Hint 3 — REM-specific guidance
13	Model buffer and cursor	Shift a suffix for insert/delete	Termios behavior is not assumed; document line mode
14	Iterate variable records	Validate reclen before advancing	Use the linux_dirent64 layout in section 15.14
15	Fetch/decode/execute	Separate code and data stacks	Bounds-check before every lw/sw
16	Decode fields by family	Table mnemonics and operand formats	Sign-extend branch fields and use P+4
17	Make pass one collect symbols	Pass two emits bytes/relocations	Reject reserved encodings and range overflow
18	Parse primary expressions first	Recursive descent gives precedence	Check signed overflow after each arithmetic operation
19	Build the language in layers	Store lines sorted by number	Reuse the editor/parser, but cap all tables
20	Make the monitor a command dispatcher	Fixed trusted routine table	Do not jr through unvalidated user bytes

Complete reference implementations, when verified, belong in `docs/assembly-programming-manual/exercises` and are intentionally not reproduced in this chapter. The accompanying `Makefile` builds only solutions whose source and target prerequisites are present; it must not claim guest or Fold execution unless a transcript is recorded.

16. Reference limitations and unresolved work

The following are intentionally not fabricated:

- The Linux syscall list is a maintained bootstrap dispatcher, not a complete generated architecture syscall table.
- Dynamic loader, GOT/PLT, shared-library, and dynamic-TLS formats are unfinished.
- Structure-return, aggregate argument, varargs save-area, signal-frame, and unwind ABI rules are not frozen.
- The exact initial auxiliary-vector contract is not yet a public stable ABI.
- Full libc coverage and all requested POSIX functions are not verified.
- Floating-point register calling conventions are not defined; current GCC support is soft-float.
- The legacy 16-bit MyEmulator ISA is a separate target.

These limitations are part of the manual because a technically useful programming reference must distinguish implemented behavior from design intent.

Appendix D. Authoritative source map

Subject	Source
Encoding manifest	docs/myemulator2-encoding.json
32-bit architecture	docs/MYEMULATOR_2_ARCHITECTURE.md
ABI and relocations	docs/MYEMULATOR_2_ABI.md
Linux machine contract	docs/MYEMULATOR_2_LINUX_PORT.md
QEMU decoder/translator	qemu/target/myemulator32/translate.c
QEMU helpers/exceptions	qemu/target/myemulator32/helper.c
GCC instruction patterns	toolchain/gcc/myemulator2.md
User linker script	toolchain/userspace/myemulator2-user.ld
Linux syscall dispatcher	linux/arch/myemulator2/kernel/syscall.c
Linux entry and exception frame	linux/arch/myemulator2/kernel/entry.S
Assembly syscall wrappers	toolchain/userspace/minilibc/syscall.S
C runtime example	toolchain/examples/linux-libc-regression.c
Direct syscall example	toolchain/examples/linux-echo.S
CPU tests	tests/run-myemulator32-*.py
Toolchain tests	toolchain/tests/test-*.py

Appendix E. Revision and reproducibility

Generate this manual from the repository checkout that supplied the source map. Record the Git revision with:

```
git rev-parse HEAD
```

The PDF is a derived artifact. The Markdown source remains the reviewable authority for this document, while the implementation and executable tests remain authoritative for processor behavior.